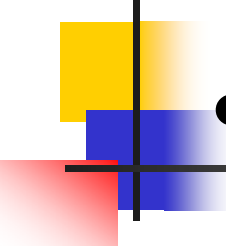


智能识别系统设计

上课地点：南区计算机大楼323

深圳大学
计算机与软件学院

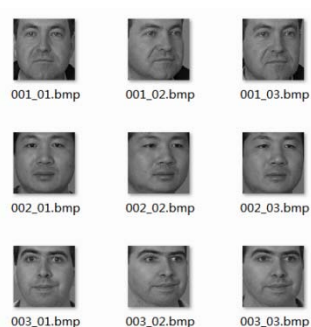


● 本次课程任务

1. AdaBoost目标检测方法的训练
2. OpenCV中的浮点矩阵运算
3. 子空间方法-主成分分析基本原理
4. 主成分分析程序设计与实现
5. 分类框架的设计
6. 基于主成分分析的视频人脸识别程序设计
7. 第二次实验报告

● AdaBoost目标检测方法的训练

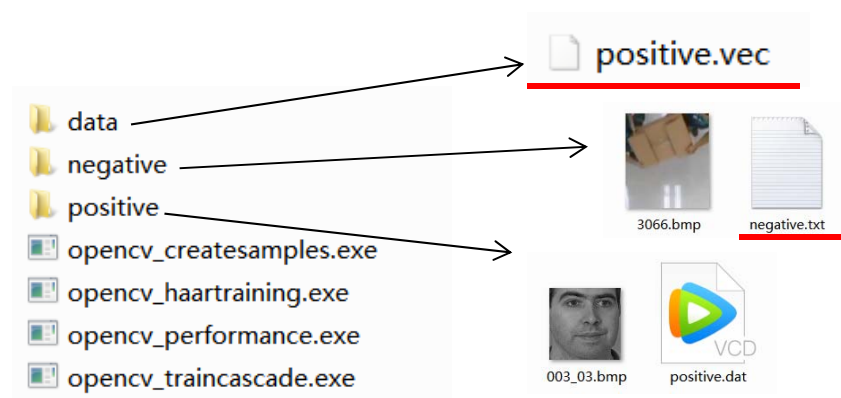
1. 准备文件



(a) 正样本图像数据



(b) 负样本图像数据



(c) 训练文件准备

2. 创建样本名字列表

3. 训练

4. 训练未完成提取XML文件的方法

● AdaBoost目标检测方法的训练

2. 创建样本名字列表

(1) 对于负样本**bmp**图片，统统放入一个文件夹**negative**生成其名字列表**negative.txt**就可以了，比如：

0.bmp
1.bmp
10.bmp
100.bmp
1000.bmp
1001.bmp
1002.bmp
1003.bmp
1004.bmp
1005.bmp
1006.bmp
...

创建命令（命令窗口下在**negative**文件夹的上一级目录键入）
`dir negative /b >negative.txt`

创建完后把**negative.txt**放入**negative**文件夹与图片一起

（负样本不需要创建**.vec**信息文件）

● AdaBoost目标检测方法的训练

2. 创建样本名字列表

(2) 对于正样本**bmp**图片，统统放入一个文件夹**positive**
生成其名字列表**positive.dat**，比如：

```
0.bmp 1 0 0 38 35
1001.bmp 1 0 0 39 43
101.bmp 1 0 0 53 50
1013.bmp 1 0 0 35 36
1015.bmp 1 0 0 38 38
1017.bmp 1 0 0 36 40
1019.bmp 1 0 0 38 38
...
```

其中“0.bmp 1 0 0 38 35”意思为：

1：图片中存在目标个数

0 0：目标在图片的左上角坐标

38 35：目标在图片的宽和高

关于如何生成名字列表：

关于名字列表的生成需要使用**Video_Image_Sample**程序，先生成正样本的名字列表，就像上面的负样本，把正样本和名字列表**.txt**文件一起放入程序目录下，通过按键**create info file**打开**.txt**，自动读取图片，读取其宽和高信息，重新生成**Info.txt**，就是上面的名字列表

把**Info.txt**内容拷入**positive.dat**，或者改名，然后放入**positive**文件夹，与图片一起

- AdaBoost目标检测方法的训练

2. 创建样本名字列表

创建.vec文件:

```
D:\Kevin\Training>"D:\Kevin\Training\opencv_createsamples.exe" -info "positive\positive.dat"  
-vec data\positive.vec -num 1000 -w 20 -h 20
```

其中:

-info "positive\positive.dat" : 是生成vec所需要的信息文件

-vec data\positive.vec : 是vec文件输出

-num 1000 : 正样本数量 (根据实际情况设置)

-w 20 -h 20: 规范图片大小 (图像中能够检测到目标的最小面积)

- AdaBoost目标检测方法的训练

3. 训练

```
D:\Kevin\Training>"D:\Kevin\Training\opencv_haartraining.exe" -data  
data\cascade -vec data\positive.vec -bg negative\negative.txt -nstages  
20 -npos 1000 -nneg 1957 -mem 1200 -mode ALL -w 20 -h 20
```

● AdaBoost目标检测方法的训练

4. 训练未完成提取XML文件的方法

nstages 17

标号从: 0到16

注意下面感叹号两关键步骤

在\temp\data\cascade文件夹中应该包含从"0"到"N"标号(N是完成的层级的标号)的目录
在每一个目录中都有一个AdaBoostCARTHaarClassifier.txt file.

!!!我们现在将0-N这些文件夹拷贝到..\cascade2xml\data catalogue文件夹

*) 使用\cascade2xml\haarconv.exe并加上参数我们就可以创建xml文件

!!!命令: haarconv.exe data output.xml 24 24

就能够创建在识别部分使用的xml文件了

你也可以使用\cascade2xml\convert.bat批处理文件来开始

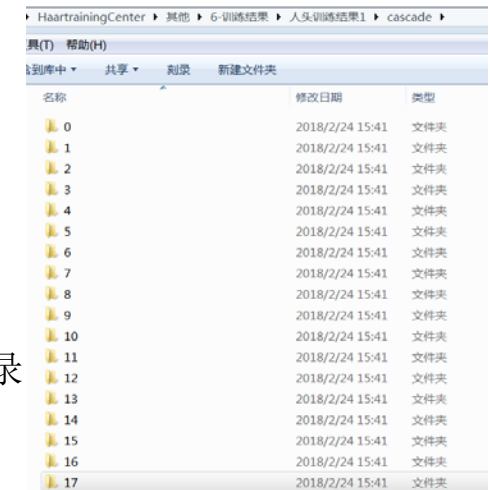
*) 将output.xml文件拷到\test_recognition文件夹

使用命令:

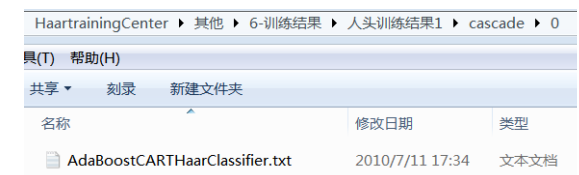
facedetect.exe --cascade="output.xml" 0

你可以使用摄像头作为图像源进行识别

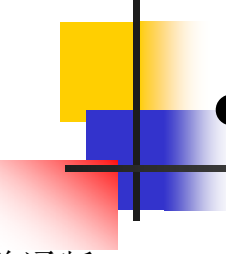
如果不用摄像头你也可以使用图像文件作为输入比如pic0.jpg或者是电影film.avi



名称	修改日期	类型
0	2018/2/24 15:41	文件夹
1	2018/2/24 15:41	文件夹
2	2018/2/24 15:41	文件夹
3	2018/2/24 15:41	文件夹
4	2018/2/24 15:41	文件夹
5	2018/2/24 15:41	文件夹
6	2018/2/24 15:41	文件夹
7	2018/2/24 15:41	文件夹
8	2018/2/24 15:41	文件夹
9	2018/2/24 15:41	文件夹
10	2018/2/24 15:41	文件夹
11	2018/2/24 15:41	文件夹
12	2018/2/24 15:41	文件夹
13	2018/2/24 15:41	文件夹
14	2018/2/24 15:41	文件夹
15	2018/2/24 15:41	文件夹
16	2018/2/24 15:41	文件夹
17	2018/2/24 15:41	文件夹



名称	修改日期	类型
AdaBoostCARTHaarClassifier.txt	2010/7/11 17:34	文本文档



● OpenCV中的浮点矩阵运算

普通版

```
double t;  
t = (double)getTickCount();  
Mat img1(1000, 1000, CV_32F); //第一个参数是高h，第二个参数是宽w；或者cvSize(w,h)或image.size()  
  
for (int i=0; i<1000; i++)  
{  
    for (int j=0; j<1000; j++)  
    {  
        img1.at<float>(i,j) = 3.2f;  
    }  
}  
t = (double)getTickCount() - t;  
printf("in %gms\n", t*1000/getTickFrequency());
```

加速版

```
t = (double)getTickCount();  
Mat img2(1000, 1000, CV_32F); //第一个参数是高h，第二个参数是宽w；或者cvSize(w,h)或image.size()  
  
for (int i=0; i<1000; i++)  
{  
    for (int j=0; j<1000; j++)  
    {  
        img2.ptr<float>(i)[j] = 3.2f;  
    }  
}  
t = (double)getTickCount() - t;  
printf("in %gms\n", t*1000/getTickFrequency());
```

● OpenCV中的浮点矩阵运算

高速版1

```
t = (double)getTickCount();  
Mat img3(1000, 1000, CV_32F); //第一个参数是高h，第二个参数是宽w；或者cvSize(w,h)或image.size()  
float* pData = (float*)img3.data;  
  
for (int i=0; i<1000; i++)  
{  
    for (int j=0; j<1000; j++)  
    {  
        *(pData) = 3.2f;  
        pData++;  
    }  
}  
t = (double)getTickCount() - t;  
printf("in %gms\n", t*1000/getTickFrequency());
```

高速版2

```
t = (double)getTickCount();  
Mat img4(1000, 1000, CV_32F); //第一个参数是高h，第二个参数是宽w；或者cvSize(w,h)或image.size()  
  
for (int i=0; i<1000; i++)  
{  
    for (int j=0; j<1000; j++)  
    {  
        ((float*)img3.data)[i*1000+j] = 3.2f;  
    }  
}  
t = (double)getTickCount() - t;  
printf("in %gms\n", t*1000/getTickFrequency());
```

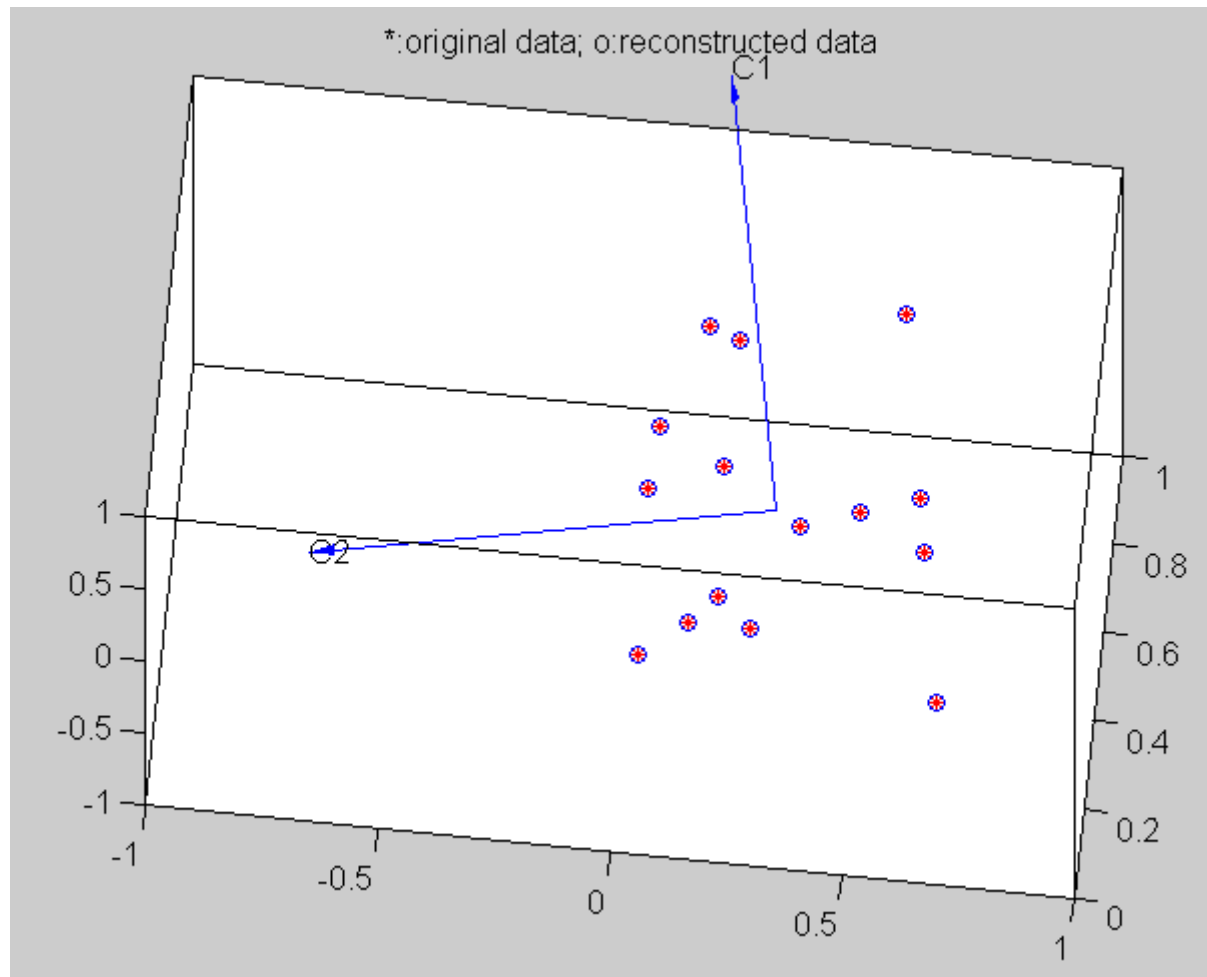

- OpenCV中的浮点矩阵运算

在Debug、Release模式下的测试结果分别为：
测试结果

	Debug	Release
普通版	139.06ms	2.51ms
加速版	66.28ms	2.50ms
高速版1	4.95ms	2.28ms
高速版2	5.11ms	1.37ms

- 子空间方法-主成分分析基本原理

主成分分析 (Principal component analysis, PCA)



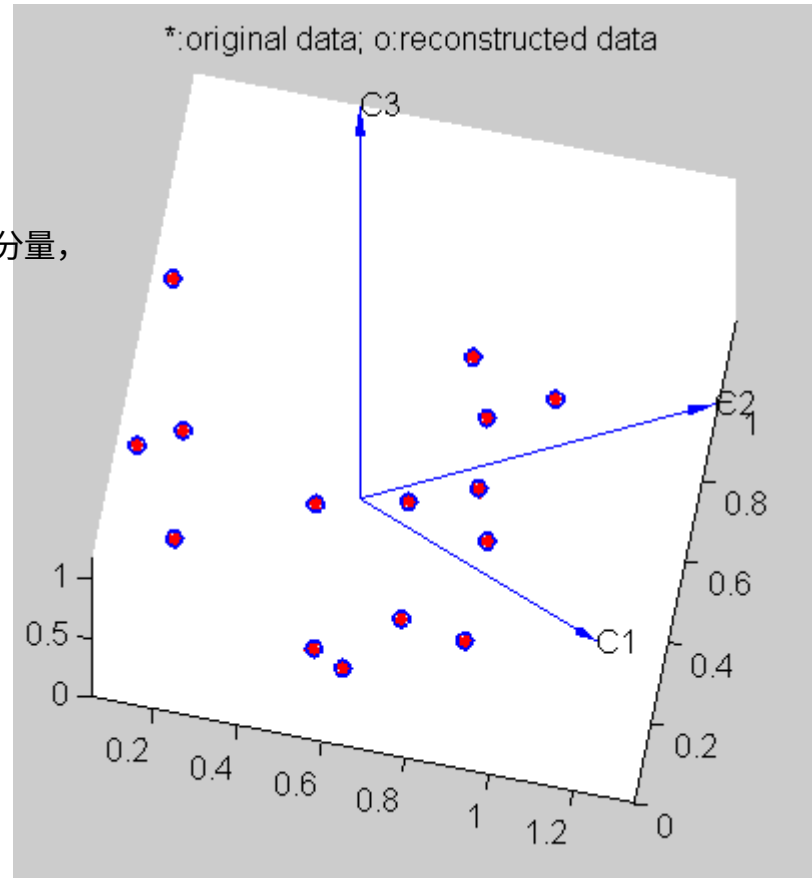
二维空间点的PCA重构

- 通过特征线性组合实现降维

- 寻找最小均方意义下最能够代表原始数据的投影方法

●子空间方法-主成分分析基本原理

在新坐标系下找出某个点的三维方向上的分量，
通过矢量相加还原表达该点



三维空间点的PCA重构

●点的维数与PCA特征轴的总数一样。

●PCA选择关键特征向量对原始数据进行重构

●**特征向量**选择原则：最小均方对应特征值。从而使**重构误差**最小

消除某一坐标轴后得到的点（即该点其他平面上的投影点），该点与投影点的距离即为重构误差

去除信息量较少的坐标轴，从而实现降维的效果（重构）

●子空间方法-主成分分析基本原理

问题：对于d维空间中的n个样本 $x_1, x_2, \dots, x_n$ ，如何能在低维空间中最好地表示它们。

(1) 0维时情况（投影到一个点）——所有点都用该点来表示，这个点在哪里？

x_0 到所有点的平方距离最小 $E_0(x_0) = \sum_{i=1}^n \|x_0 - x_i\|^2$ ，其中 $m = \frac{1}{n} \sum_{i=1}^n x_i$

$$\begin{aligned} E_0(x_0) &= \sum_{i=1}^n \|(x_0 - m) - (x_i - m)\|^2 \\ &= \sum_{i=1}^n \|x_0 - m\|^2 - 2 \sum_{i=1}^n (x_0 - m)^T (x_i - m) + \sum_{i=1}^n \|x_i - m\|^2 \\ &= \sum_{i=1}^n \|x_0 - m\|^2 - 2(x_0 - m)^T \sum_{i=1}^n (x_i - m) + \sum_{i=1}^n \|x_i - m\|^2 \end{aligned}$$

$$\sum_{i=1}^n (x_i - m) = \sum_{i=1}^n x_i - nm = \sum_{i=1}^n x_i - n \cdot \frac{1}{n} \sum_{i=1}^n x_i = 0$$

所以有 $E_0(x_0) = \sum_{i=1}^n \|x_0 - m\|^2 + \sum_{i=1}^n \|x_i - m\|^2$ 即当 x_0 等于均值时，有最小均方

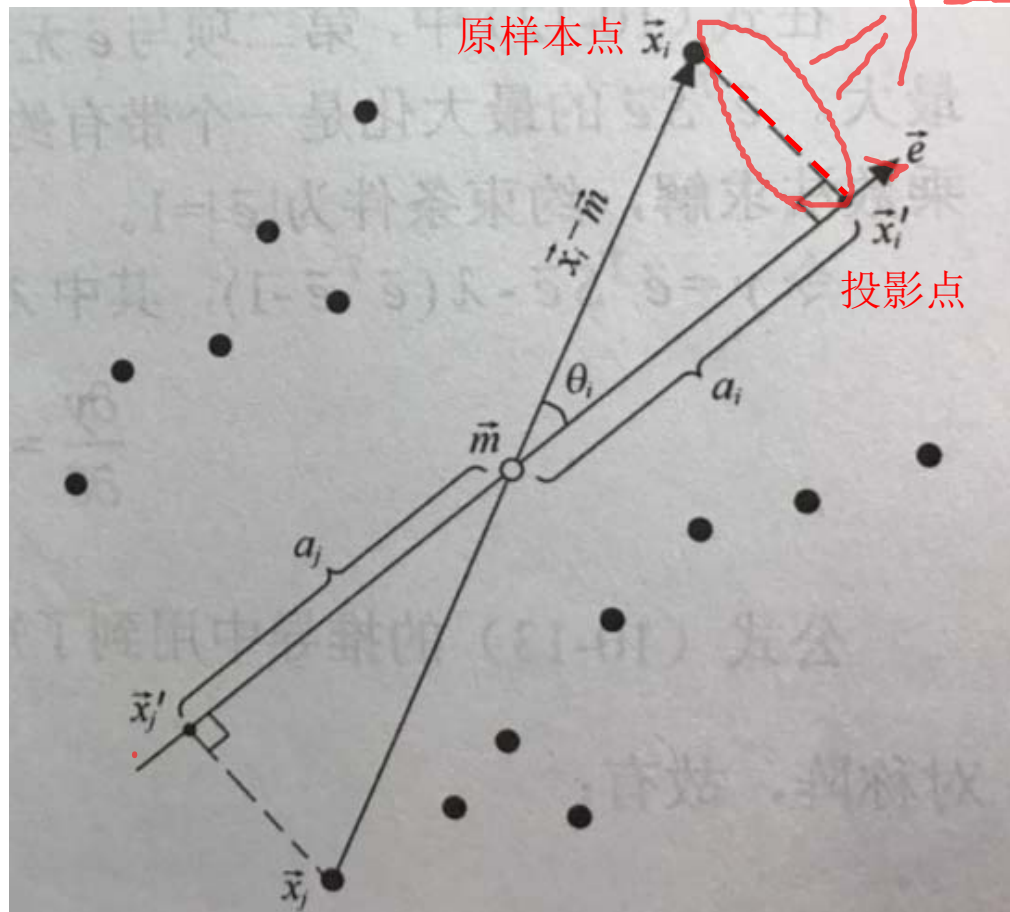
所有样本在0维空间都压缩到一个点，无法反映样本之间的差异。

平均点

为零定值

●子空间方法-主成分分析基本原理

(2) 1维时情况（投影到一条直线 e ）



设 $E_1(e)$

过 m 的直线方程

$$x = m + ae$$

$$a_i = |x_i - m| \cdot |e| \cdot \cos(\theta) = e \cdot (x_i - m) = e^T (x_i - m)$$

如何确定最优 e 方向使 $E_1(e)$ 最小

即 e 方向确定以后，所有样本点投影到这个直线产生投影点。这些投影点到原样本点的距离之和即为 $E_1(e)$

样本的一维表达

●子空间方法-主成分分析基本原理

$$E_1(e) = \sum_{i=1}^n \|x - x_i\|^2 = \sum_{i=1}^n \|m + a_i e - x_i\|^2 = \sum_{i=1}^n \|a_i e - (x_i - m)\|^2$$

$$= \sum_{i=1}^n a_i^2 \|e\|^2 - 2 \sum_{i=1}^n a_i e^T (x_i - m) + \sum_{i=1}^n \|x_i - m\|^2$$

Handwritten note: $x_i - m = a_i e$

$$E_1(e) = \sum_{i=1}^n a_i^2 - 2 \sum_{k=i}^n a_i^2 + \sum_{i=1}^n \|x_i - m\|^2$$

$$= - \sum_{i=1}^n a_i^2 + \sum_{i=1}^n \|x_i - m\|^2$$

$$= - \sum_{i=1}^n [e^T (x_i - m)]^2 + \sum_{i=1}^n \|x_i - m\|^2$$

$$= - \sum_{i=1}^n e^T (x_i - m) (x_i - m)^T e + \sum_{i=1}^n \|x_i - m\|^2$$

$$= - e^T \left[\sum_{i=1}^n (x_i - m) (x_i - m)^T \right] e + \sum_{i=1}^n \|x_i - m\|^2$$

$$= - e^T S e + \sum_{i=1}^n \|x_i - m\|^2$$

S 为散布矩阵，是样本协方差的n-1倍

令 $\|e\| = 1$

$$y = e^T S e - \lambda (e^T e - 1)$$

求导得 $S e = \lambda e$ e 是 S 特征向量时最大

S 矩阵在特征向量 e 中的特征值

$$e^T S e = \lambda e^T e = \lambda$$

特征值越大对应特征向量对重建效果最好

●子空间方法-主成分分析基本原理

(3) 多维时情况 (投影到多条直线 e_t)

$$x = m + \sum_{k=1}^t a_k e_k, t \leq d$$

多个特征向量

样本点原始维数

$$E_t(e_1, e_2, \dots, e_t) = \sum_{i=1}^n \left\| \left(m + \sum_{k=1}^t a_{ik} e_k \right) - x_i \right\|^2$$

$e_k (k=1, 2, \dots, t)$ 分别对应 S 的前 t 个特征值 (从大到小) 对应的特征向量

●子空间方法-主成分分析基本原理

只适用于样本点维数较低的情况

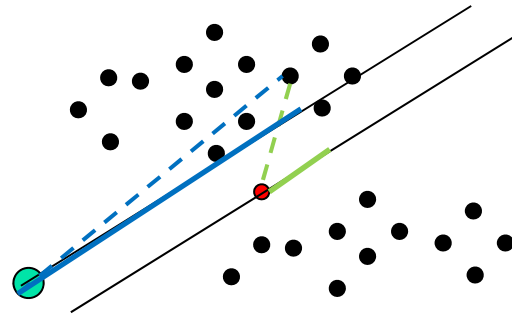
PCA算法原理

输入数据矩阵 X (维数 $d \times n$), 其中 d 是样本维数,
 n 是样本个数

1. 求样本均值 m (维数 $d \times 1$), 即对 X 中所有样本求均值;
2. 计算中心化的数据 X_m (维数 $d \times n$), 即 X 中的每一个样本都要减去均值 m , 形成的数据矩阵;
3. 计算协方差矩阵 S (维数 $d \times d$)= $X_m \cdot X_m^T$, 其中去中心化数据矩阵 X_m 维数是 $d \times n$;
4. 对 S 进行特征分解, 获得前 NumAxes 个最大特征值, 及对应特征向量;
5. 对每一个特征向量, 把他们归一化, 即每个元素除以向量的2范数 (所有获得的特征向量构成特征向量矩阵 P , 维数 $d \times \text{NumAxes}$);
6. 把去中心化的数据 X_m (维数 $d \times n$), 投影到特征向量矩阵 P (维数 $d \times \text{NumAxes}$) 获得低维表示 T (维数 $n \times \text{NumAxes}$)= $X_m^T \cdot P$.

排序
调用库

- 子空间方法-主成分分析基本原理



●子空间方法-主成分分析基本原理

降维数
↓

PCA算法实现要注意的地方:

1. 输入部分: 所有训练样本图像和测试样本图像都要归一化到统一的大小, 再拉成列向量(维数 $d*1$), 数据矩阵 X (维数 $d*n$)的每一列就是由所有样本对应列向量构成;
2. 在第4步对散布矩阵进行特征分解后, 要对特征值从大到小进行排序, 对应特征向量的排序相应发生变化; 对于特征值很小或者为0的特征向量可以忽略, 也即最终选择特征向量的个数 $NumAxes \leq d$;
3. 低维表示 T (维数 $n*NumAxes$)中的每一行表示每一个样本(维数 $d*1$)
在低维空间的表示 $t(1*NumAxes)$, 在算法中最好把 T 和 t 都转置形成标准的向量表示格式。这样, 原来 d 维的样本, 在子空间中就变成 $NumAxes$ 维的向量, 实现了降维和子空间表示的目的, 其中 $NumAxes \leq d$ 。

●子空间方法-主成分分析基本原理

散布矩阵 $S=X_m X_m'$ 维数 $d*d$ ，当维数很高时，计算非常耗时

考虑 $R=X_m' X_m$ 维数 $n*n$ ，在识别问题中 n 远远小于 d

令 v 是 R 的特征向量

降维系数

所以有 $X_m^T X_m v = \lambda v$

$$X_m X_m^T X_m v = \lambda X_m v$$

$$S(X_m v) = \lambda (X_m v)$$

也即 R 的特征向量左乘 X_m 即得到 S 的特征向量，但特征值分解计算量大大减小

●子空间方法-主成分分析基本原理

PCA算法原理

输入数据矩阵 X (维数 $d*n$), 其中 d 是样本维数, n 是样本个数

1. 求样本均值 m (维数 $d*1$), 即对 X 中所有样本求均值;
2. 计算中心化的数据 X_m (维数 $d*n$), 即 X 中的每一个样本都要减去均值 m , 形成的数据矩阵;
3. 计算协方差矩阵 S (维数 $d*d$)= $X_m * X_m'$, 其中去中心化数据矩阵 X_m 维数是 $d*n$;
4. 对 S 进行特征分解, 获得前 $NumAxes$ 个最大特征值, 及对应特征向量;
5. 对每一个特征向量, 把他们归一化, 即每个元素除以向量的2范数 (所有获得的特征向量构成特征向量矩阵 P , 维数 $d*NumAxes$);
6. 把去中心化的数据 X_m (维数 $d*n$), 投影到特征向量矩阵 P (维数 $d*NumAxes$) 获得低维表示 T (维数 $n*NumAxes$)= $X_m' * P$ 。

快速PCA算法原理

输入数据矩阵 X (维数 $d*n$), 其中 d 是样本维数, n 是样本个数

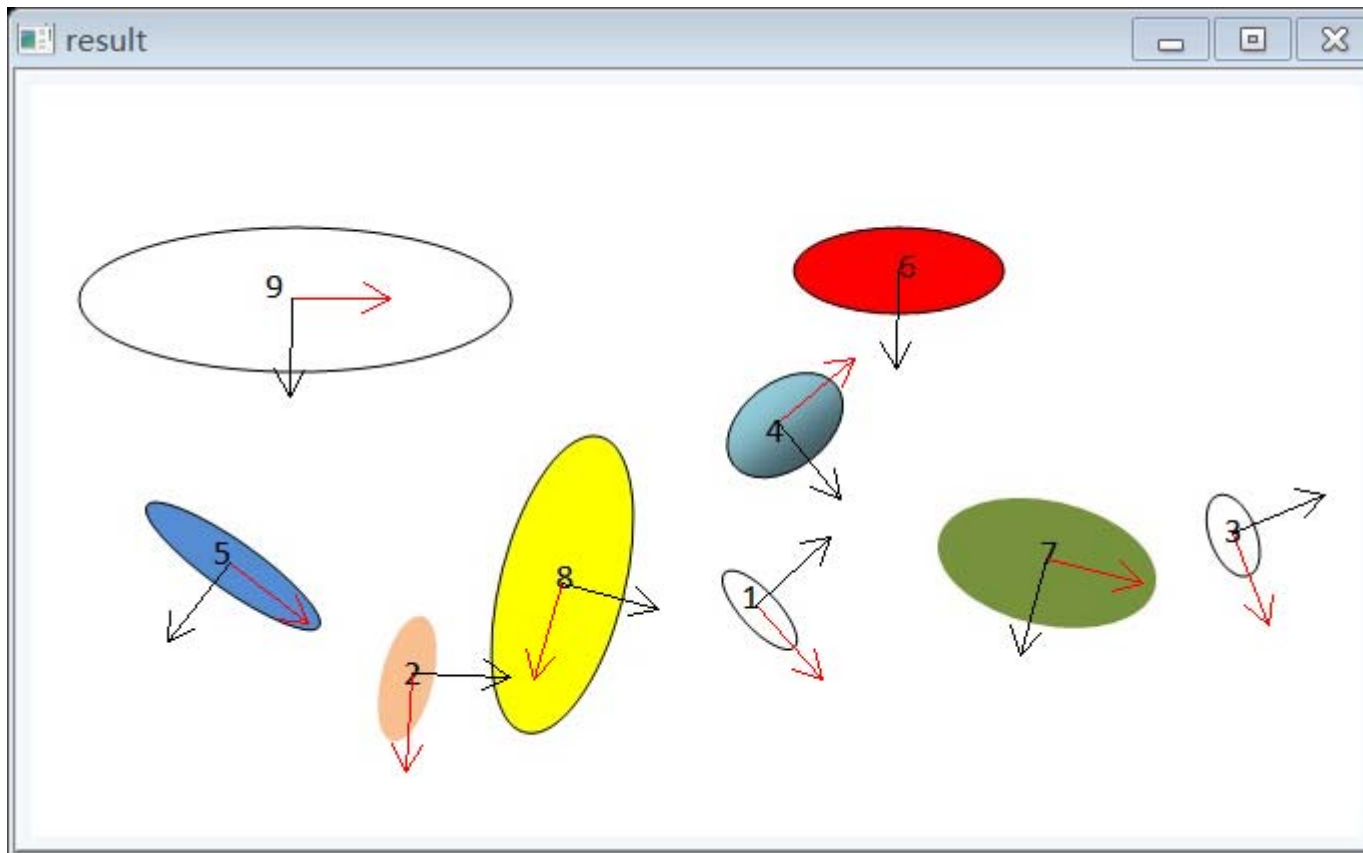
1. 求样本均值 m (维数 $d*1$), 即对 X 中所有样本求均值;
2. 计算中心化数据 X_m ($d*n$), 即 X 中的每一个样本都要减去均值 m , 形成的数据矩阵;
3. 计算 R 矩阵(维数 $n*n$)= $X_m' * X_m$, 其中去中心化数据矩阵 X_m 维数是 $d*n$;
4. 对 R 矩阵进行特征分解, 获得前 $NumAxes$ 个最大特征值, 及对应特征向量, 则可以获得特征向量矩阵 P (维数 $NumAxes*n$);
5. 对特征向量矩阵 P (维数 $NumAxes*n$) 左乘 X_m ($d*n$) 得到 S 的特征向量 V (维数 $d*NumAxes$), 即 $V = X_m * P'$;
6. 对每一个特征向量, 把他们归一化, 即每个元素除以向量的2范数 (所有获得的特征向量构成特征向量矩阵 V , 维数 $d*NumAxes$);
7. 把去中心化的数据 X_m (维数 $d*n$), 投影到特征向量矩阵 V (维数 $d*NumAxes$) 获得低维表示 T (维数 $n*NumAxes$)= $X_m' * V$ 。

●子空间方法-主成分分析基本原理

快速PCA算法实现要注意的地方：

1. 输入部分：所有训练样本图像和测试样本图像都要归一化到统一的大小，再拉成列向量(维数 $d*1$)，数据矩阵 X (维数 $d*n$)的每一列就是由所有样本对应列向量构成；
2. 在第4步对散布矩阵进行特征分解后，要对特征值从大到小进行排序，对应特征向量的排序相应发生变化；对于特征值很小或者为0的特征向量可以忽略，也即最终选择特征向量的个数 $NumAxes \leq n$ ；
3. 低维表示 T (维数 $n*NumAxes$)中的每一行表示每一个样本（维数 $d*1$ ）在低维空间的表示 $t(1*NumAxes)$ ，在算法中最好把 T 和 t 都转置形成标准的向量表示格式。这样，原来 d 维的样本，在子空间中就变成 $NumAxes$ 维的向量，实现了降维和子空间表示的目的，其中 $NumAxes \leq n$ 。

●子空间方法-主成分分析基本原理



●主成分分析程序设计与实现

变量定义 {

```
typedef struct PCAStruct
```

```
//处理的数据，维数Dim*SampleSize，
```

```
CvMat *data;
```

```
//样本量大小
```

```
int SampleSize;
```

```
//一个样本的总维数（总属性个数/总正交轴个数）
```

```
int Dim;
```

```
//期望得到的正交轴个数（前n个主成份轴）
```

```
int AxesNum;
```

```
//数据的均值，维数Dim*1
```

```
CvMat *mean_vector;
```

```
//数据的协方差阵，维数Dim*Dim
```

```
CvMat *matrix_scatter;
```

```
//反馈得到的特征向量，维数Dim*AxesNum
```

```
CvMat *Eigen_vectors;
```

```
//反馈得到的每个原始样本在前AxesNum个轴上的投影，为主成份，
```

```
CvMat *Coefficients;
```

```
//反馈得到的特征值，维数AxesNum*1
```

```
CvMat *Eigen_values;
```

```
//重建的样本，维数Dim*SampleSize
```

●主成分分析程序设计与实现

类与函数

```
] class CPCAMethod
```

```
{
```

```
public:
```

```
    //构造函数
```

```
    CPCAMethod(void);
```

```
    //析构函数
```

```
    ~CPCAMethod(void);
```

```
public:
```

```
] //函数名: centralized_data
```

```
    //函数功能: 把原始数据中心化
```

```
    //参数说明:
```

```
    //CvMat *data: 输入数据, 维数Dim*SampleSize
```

```
    //CvMat *mean_vector: 均值向量, 维数 Dim*1
```

```
    //CvMat* dataC: 中心化数据, 维数Dim*SampleSize
```

```
- //无返回值
```

```
void centralized_data(CvMat *data, CvMat * mean_vector, CvMat* dataC);
```

```
] //函数名: IniPCAStruct
```

```
    //函数功能: 初始化PCADData, 传入data数据和求出均值向量
```

```
    //参数说明:
```

```
    //PCAStruct &PCADData: PCA变量结构体
```

```
    //float *data: 输入数据, 维数Dim*SampleSize
```

●主成分分析程序设计与实现

```
//                                     Dim*SampleSize          Dim*1          Dim*Sar
void CPCAMethod::centralized_data(CvMat *data, CvMat * mean_vector,CvMat* dataC
{
    int step_ori=data->step/sizeof(float);
    float * data_ori=data->data.fl;

    int step_m=mean_vector->step/sizeof(float);
    float *data_m=mean_vector->data.fl;

    int step_c=dataC->step/sizeof(float);
    float *data_c=dataC->data.fl;

    for(int i=0;i<data->cols;i++)
    {
        for(int j=0;j<data->rows;j++)
        {
            (data_c+j*step_c)[i]=(data_ori+j*step_ori)[i]-(data_m+j*step_m)[0];//访问CvMat
        }
    }
}
```

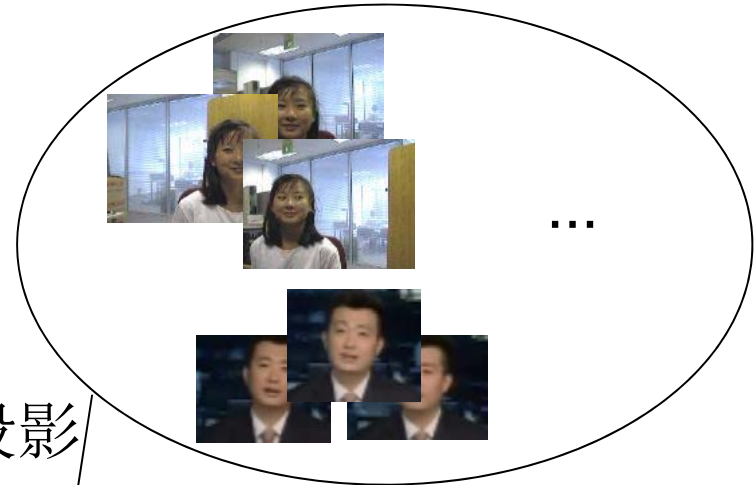
● 分类框架的设计

基于快速PCA算法分类方法

已知：每一类别的训练样本，假设有C类，每一类有k个样本，共有 $n=C*k$ 个；并已知每一类的标签1,2,3...C。

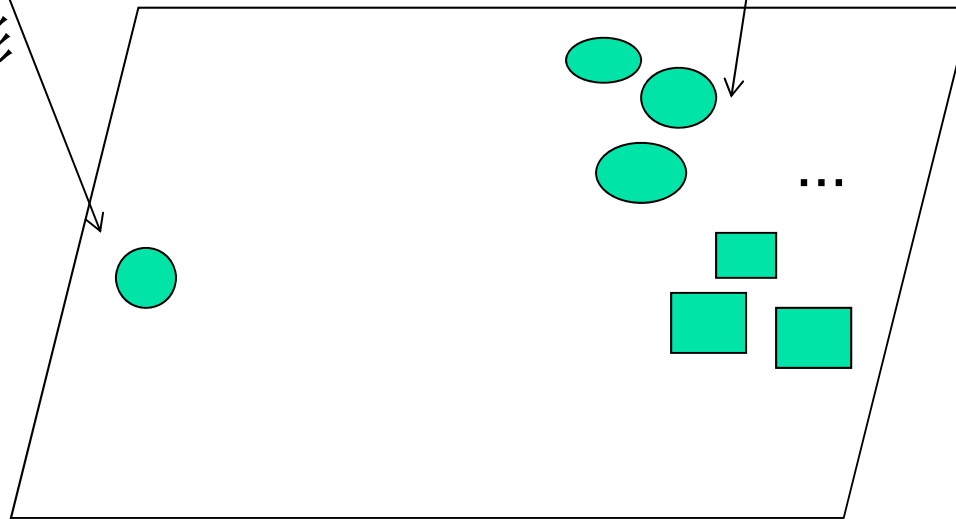
1. 利用快速PCA算法先在训练样本上计算子空间的投影向量矩阵V(维数 $d*NumAxes$)，那么对于任意一个样本s(维数 $d*1$)，通过投影得到在子空间中的表示 $s'*V=u$ (维数 $1*NumAxes$)；同时获得训练样本在子空间的表示矩阵T(维数 $n*NumAxes$)；
2. 循环以下操作
3. 给定一个测试样本t1，计算它在子空间的投影u1；
4. 分类法1：采用最近邻分类方法，u1离哪个子空间中的训练样本最近，就划分到哪一类。根据分类结果与已知标签对比统计正确率。
5. 分类法2：计算子空间中每一类训练样本的均值，u1离哪一类的距离最近，就把它划分到哪一类。根据分类结果与已知标签对比统计正确率。
6. 结束循环

● 分类框架的设计



PCA投影

PCA投影



● 基于主成分分析的视频人脸识别程序设计

训练



人脸检测

1

图像归一化

2

投影

PCA子空间

3

特征存储
到本地硬盘
测试时要
加载特征

4

测试



人脸检测

图像归一化

投影矩阵

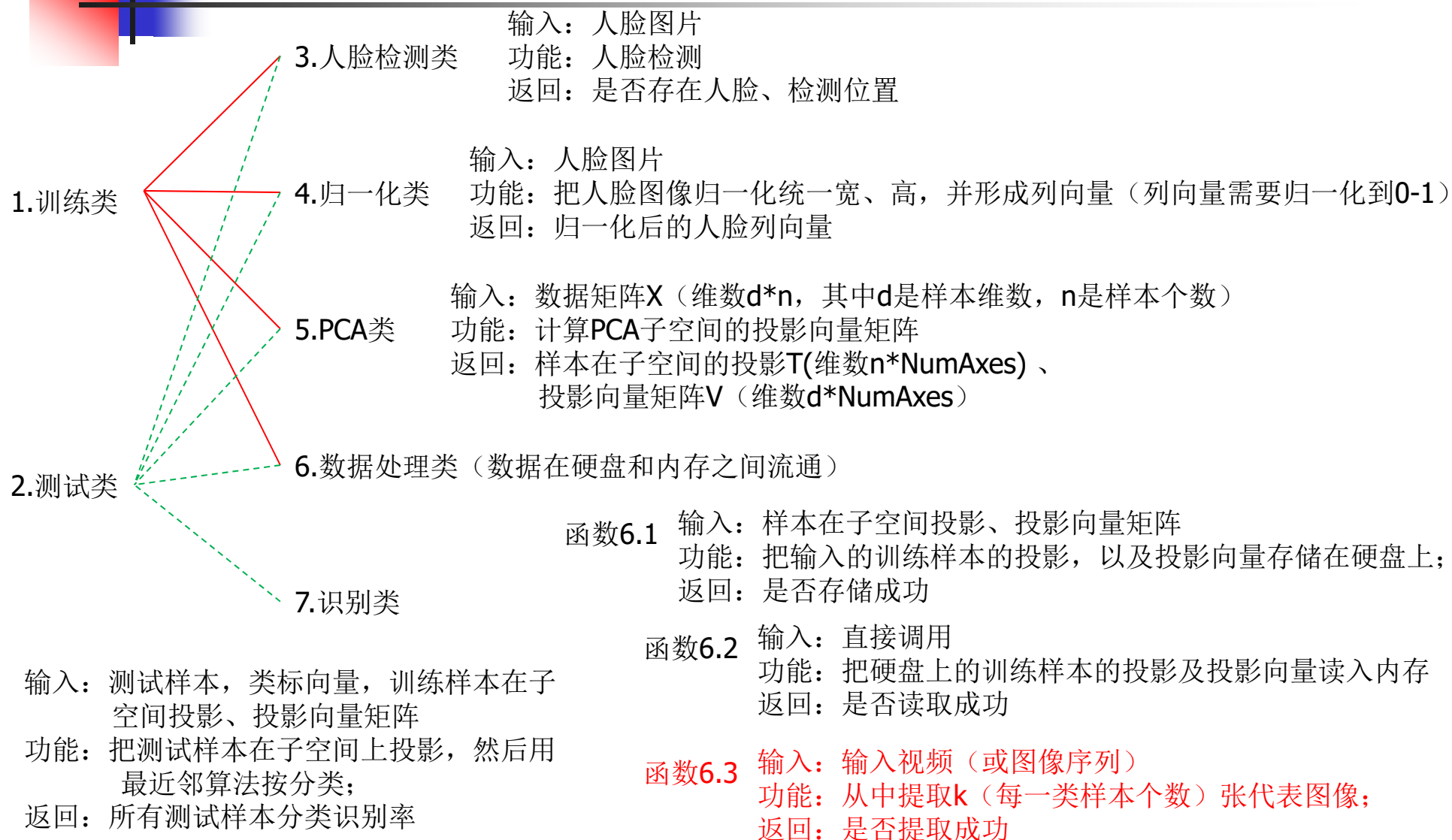
5

测试样本在
子空间表示

6

分类

● 基于主成分分析的视频人脸识别程序设计

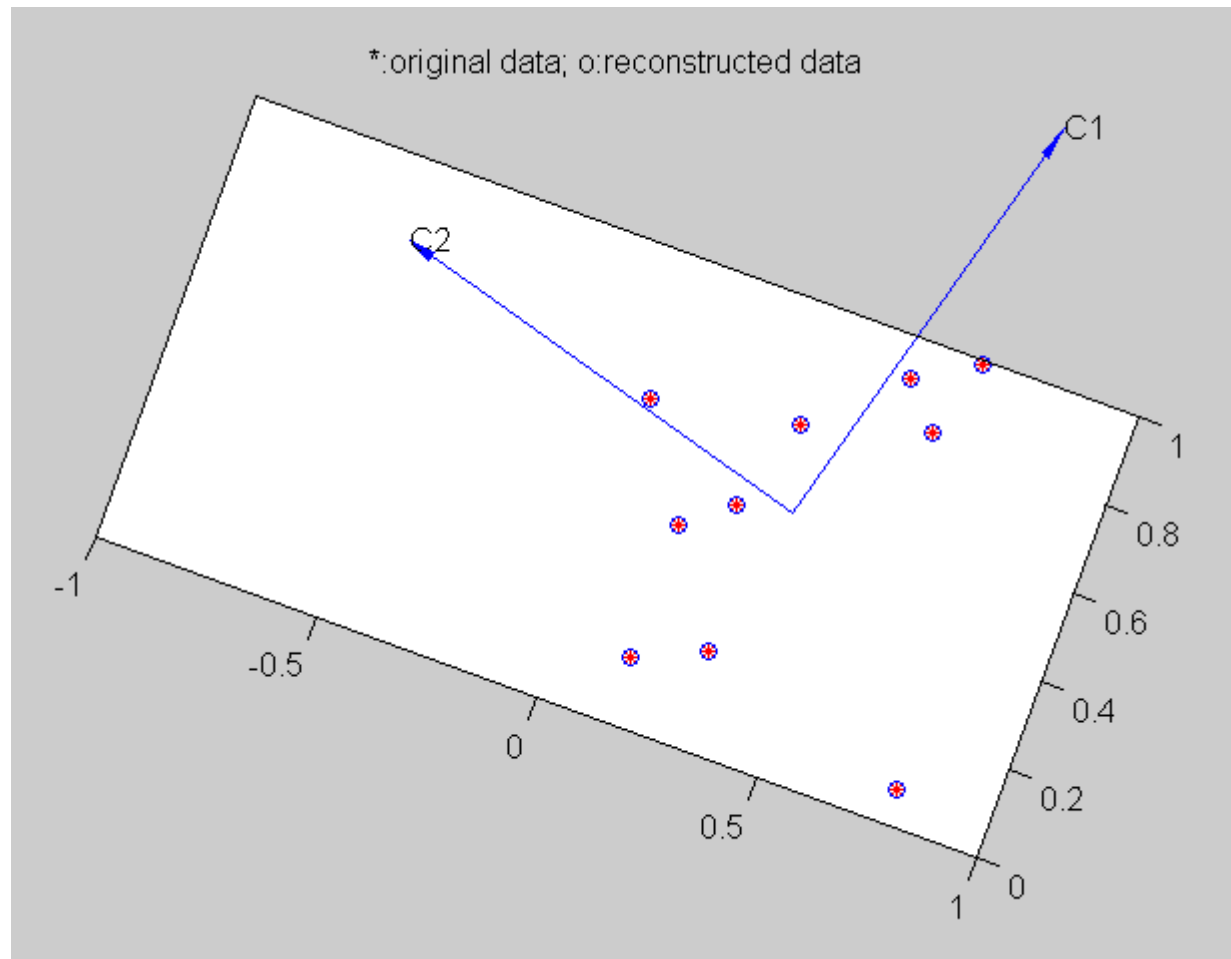


● 基于主成分分析的视频人脸识别程序设计

基本PCA测试数据

X	Y
0.01	0.68
0.33	0.74
0.16	0.45
0.79	0.08
0.31	0.22
0.52	0.91
0.16	0.15
0.6	0.82
0.26	0.53
0.65	0.9

meanvector =
0.379 0.548



- 基于主成分分析的视频人脸识别程序设计

去中心化后的数据

$X_m =$

-0.369	0.132
-0.049	0.192
-0.219	-0.098
0.411	-0.468
-0.069	-0.328
0.141	0.362
-0.219	-0.398
0.221	0.272
-0.119	-0.018
0.271	0.352

- 基于主成分分析的视频人脸识别程序设计

Eigenvalues =

0.901890466313468

0.540759533686533

Eigenvectors =

0.256342603861391

0.966585986576239

-0.966585986576239

0.256342603861391

- 基于主成分分析的视频人脸识别程序设计

快速PCA测试数据, $d=4$, $n=3$

```
x = [ 0.23  0.35  0.82  
      0.01  0.04  0.16  
      0.64  0.73  0.64  
      0.45  0.54  0.29];
```

```
%Eigenvectors: n*n  
rr=rank(R)  
if NumAxes>rr %  
    NumAxes=rr;  
end
```

meanvector =

```
0.4666666666666667  
    0.07  
    0.67  
0.4266666666666667
```

● 基于主成分分析的视频人脸识别程序设计

$X_m =$

-0.236666666666667	-0.116666666666667	0.353333333333333
-0.06	-0.03	0.09
-0.03	0.0599999999999999	-0.03
0.0233333333333333	0.113333333333333	-0.136666666666667

$R =$

0.0610555555555555	0.0302555555555555	-0.0913111111111111
0.0302555555555555	0.0309555555555555	-0.0612111111111111
-0.0913111111111111	-0.0612111111111111	0.152522222222222

● 基于主成分分析的视频人脸识别程序设计

REigenvectors =

-0.489964510815981	0.653147337748172	0.577350269189625
-0.320659931496101	-0.750895382193539	0.577350269189627
0.810624442312081	0.09774804444537	0.577350269189626

DiagEigenvalues =

0.231926631000866	0	0
0	0.0126067023324677	0
0	0	-4.37832480216799e-17

● 基于主成分分析的视频人脸识别程序设计

对特征向量矩阵P（维数NumAxes*n）左乘 $X_m(d*n)$ 得到S
的特征向量V(维数d*NumAxes)，即 $V=X_m*P'$;

$X_m =$

-0.2366666666666667	-0.1166666666666667	0.3533333333333333
-0.06	-0.03	0.09
-0.03	0.0599999999999999	-0.03
0.0233333333333333	0.1133333333333333	-0.1366666666666667

$P^T =$

-0.489964510815981	0.653147337748172
-0.320659931496101	-0.750895382193539
0.810624442312081	0.09774804444537

Eigenvectors =

0.439789229184596	-0.032436099640457
0.111973868401929	-0.00786465479900085
-0.0288593938346491	-0.0675805843974186
-0.158559304604582	-0.0832202715086777

● 视频人脸数据库介绍



共10个类别，每个类在train文件夹中选10张作为训练对象

● 第二次试验报告

视频人脸检测及识别系统设计



1.实验报告主题： 视频人脸检测及识别设计

2.实验目的：

- 1) 编程读取人脸视频或图像序列并实现对人脸的检测与预处理（提示：人脸检测方法可采用**AdaBoost**、人脸关键点提取等机器学习方法）；
- 2) 编写**PCA**或其他特征提取算法对人脸图像进行特征提取；
- 3) 设计并构建人脸识别框架。

3.实验要求：

- 1) 熟悉**OpenCV**编程环境或其他图像处理工具包；
- 2) 熟练掌握基本图像处理方法和**PCA**子空间学习等人脸特征提取方法；
- 3) 熟悉最近邻等图像分类方法；
- 4) 编程语言推荐使用**C++**，但不限于**C++**（例如可使用**Python**或**Matlab**语言）；
- 5) 无需编写界面，代码写成函数，方便调用（若以**C++**为编程语言，建议以类的形式把相关函数统一起来，方便后续使用）。

● 第二次试验报告

视频人脸检测及识别系统设计



实验报告内容：

1) 从视频或图像序列中创建训练样本，其中类别个数为10，每类样本有10个人脸图像；

2) 采用PCA或其他人脸特征提取方法提取人脸特征，并存储到文本文件中（提示：若用PCA方法，则需要把训练样本投影到子空间，获得训练样本在子空间的投影以及投影矩阵），以便测试时调用；

3) 对10个类别的测试视频或测试图像序列进行测试，对于每一个类，输入其测试视频或测试图像序列，然后通过人脸检测、归一化、分类等一系列方法实现对视频或图像序列中人脸的识别，并统计识别率，若10个测试视频（或图像序列）都识别正确，则识别率为100%。

提交时间：2022年11月05日下午5:00